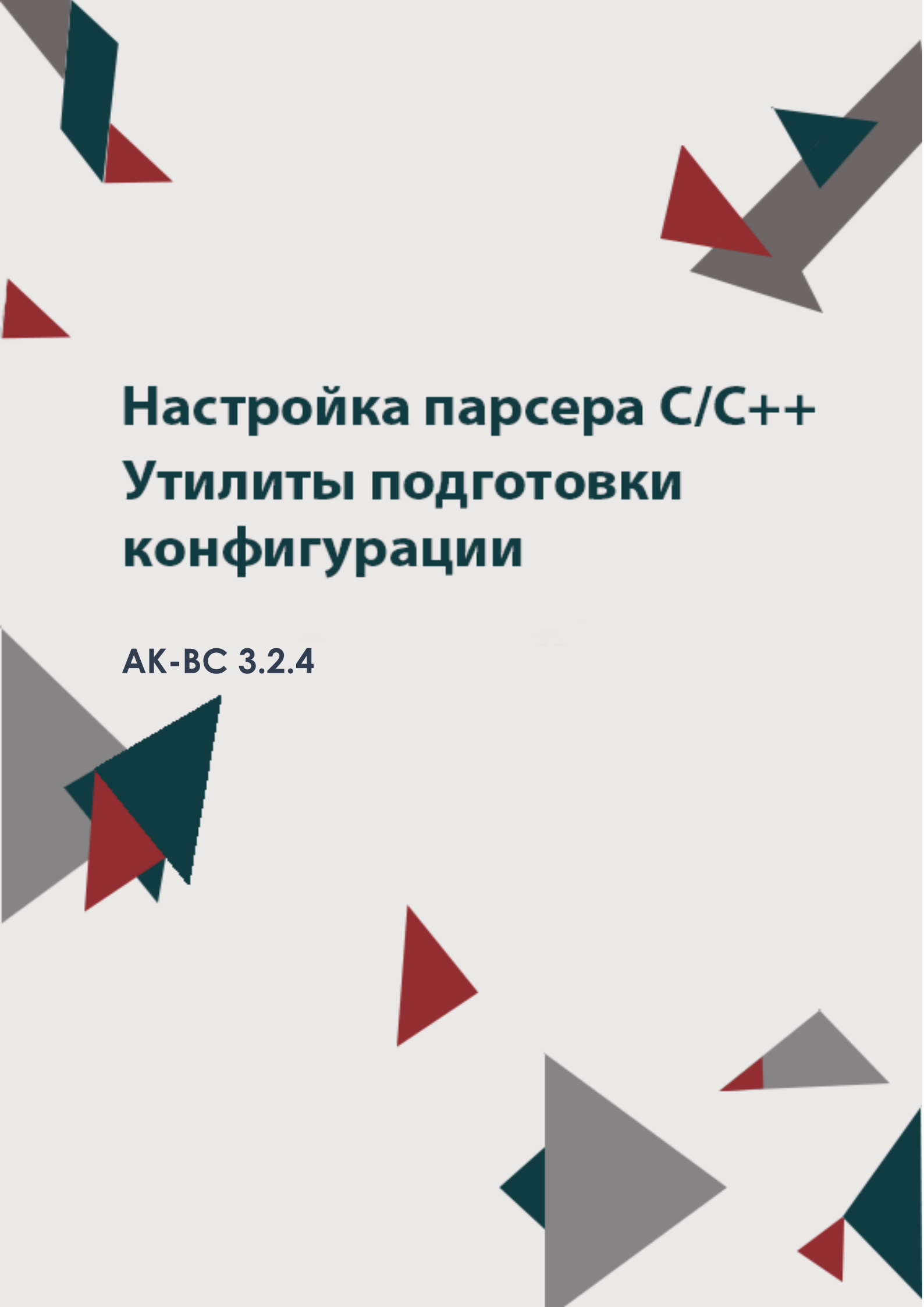


The background of the page is decorated with various abstract geometric shapes, including triangles and polygons, in shades of teal, red, and grey. These shapes are scattered across the page, with some appearing as large, prominent elements and others as smaller, more subtle accents. The overall aesthetic is modern and minimalist.

Настройка парсера C/C++ Утилиты подготовки конфигурации

АК-ВС 3.2.4

Введение

Программа на языках C/C++ состоит из множества файлов, каждый из которых может компилироваться отдельно. При этом содержание программы зависит от аргументов каждого вызова компилятора. В частности, на содержание влияют директивы препроцессора.

Поэтому необходимо настроить проект, чтобы сообщить анализатору об аргументах каждого вызова компилятора и дополнительных используемых заголовочных файлах.

Настройка проекта включает в себя:

- ❖ указание путей до заголовочных файлов;
- ❖ загрузку сторонних заголовочных файлов (например, от используемых библиотек) и указание путей до них;
- ❖ указание значений макросов;
- ❖ указание прочих параметров компиляции (например, используемых стандартов языка).

Рассмотрим пример того, что произойдет, если не выполнить настройку:

файл **a.h**:

```
1 #define MY_IF(a) if(a) {
```

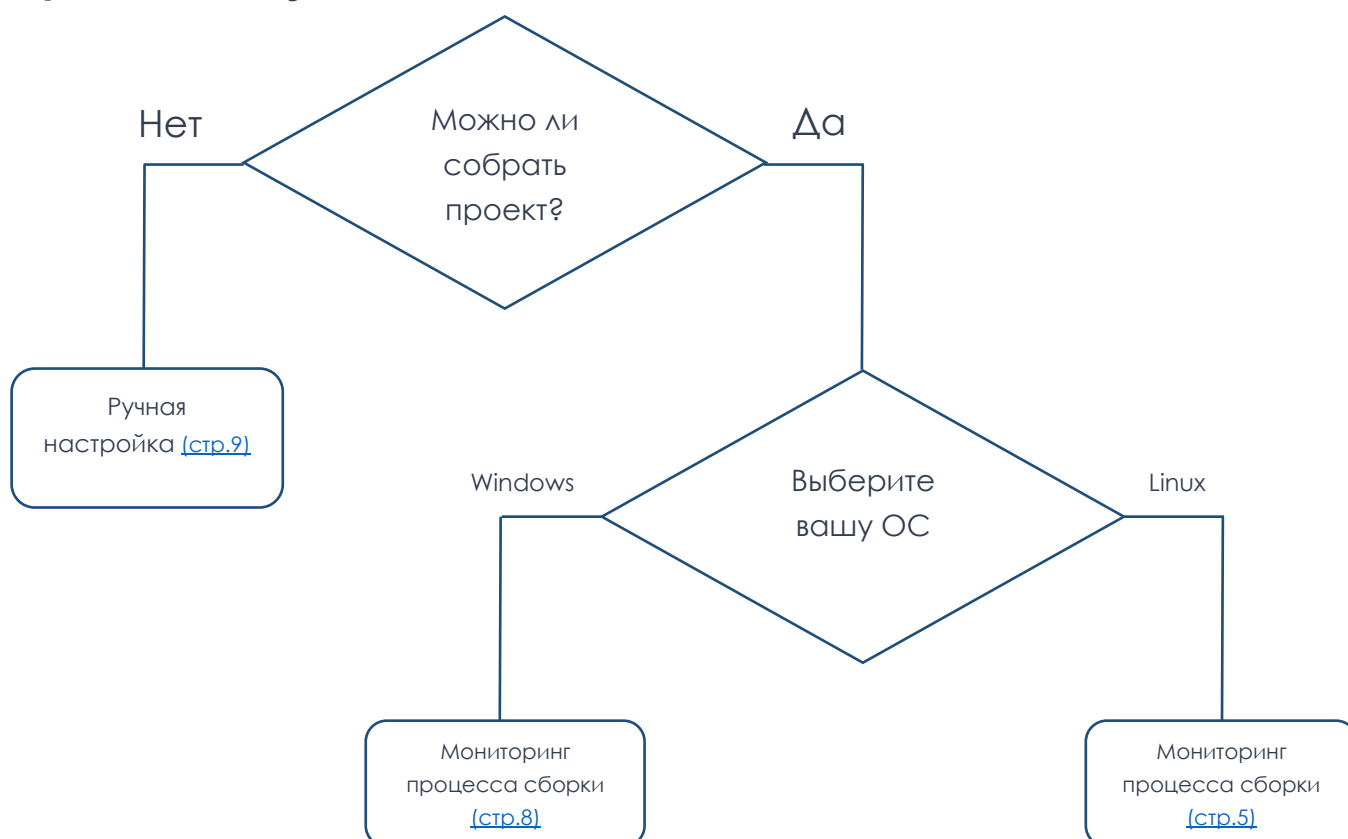
Файл **a.cpp**:

```
1 #include "a.h"
2
3 int main() {
4     int b = 1;
5     MY_IF(b)
6         b++;
7 }
8
9 my_function();
10 }
```

Если в процессе анализа файл **a.h** не будет найден, то парсер примет за конец функции **main** фигурную скобку на седьмой строке вместо десятой.

В комплекте с АК-ВС 3 идут две утилиты, которые помогут сконфигурировать проект в автоматическом режиме.

Применение утилит



СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
ПРИМЕНЕНИЕ УТИЛИТ	3
СОДЕРЖАНИЕ	4
УТИЛИТЫ АВТОМАТИЧЕСКОЙ КОНФИГУРАЦИИ	5
ПОДГОТОВКА ИСХОДНЫХ ТЕКСТОВ К АНАЛИЗУ	5
УТИЛИТА AKVS-BUILD-MONITOR АВТОМАТИЧЕСКОЙ КОНФИГУРАЦИИ ДЛЯ ПРОЕКТОВ НА C/C++ И JAVA В СРЕДЕ LINUX.....	5
УТИЛИТА SRRCNF_LINUX.PY АВТОМАТИЧЕСКОЙ КОНФИГУРАЦИИ ДЛЯ ПРОЕКТОВ НА C/C++ В СРЕДЕ LINUX.....	7
УТИЛИТА АВТОМАТИЧЕСКОЙ КОНФИГУРАЦИИ ДЛЯ ПРОЕКТОВ НА C/C++ В СРЕДЕ WINDOWS	9
РУЧНАЯ НАСТРОЙКА	11
ФОРМАТ КОНФИГУРАЦИОННОГО ФАЙЛА	11
ОБЩИЕ НАСТРОЙКИ ПАРСЕРА	11
ОПЦИИ ПАРСЕРА ДЛЯ КОМПИЛЯТОРОВ И СТАНДАРТОВ ЯЗЫКА C++	13
АРГУМЕНТЫ ПАРСЕРА ДЛЯ ОТДЕЛЬНЫХ ФАЙЛОВ	13
СИНТЕТИЧЕСКИЙ ПРИМЕР	16

Утилиты автоматической конфигурации

Подготовка исходных текстов к анализу

Понять, что является исходными текстами программы, можно, только анализируя сборочные скрипты. Сборка является сложным процессом, выполняемым специализированными программами, такими, как `autotake`, `conf`, `cmake`, `maven` и т.п., на основании файлов проекта, или скриптами произвольной формы. Поэтому АК-ВС 3 собирает информацию о составе исходных текстов проекта, наблюдая за реально выполняемым процессом сборки. Основное требование для успешного перехвата сборки – это должна быть сборка приложения «с нуля». Повторная сборка зачастую пропускает те файлы, которые уже были созданы. Поскольку штатные средства очистки сборки (`make clean`) пишут разработчики, они могут содержать ошибки и не всегда срабатывать правильно. Это нужно учитывать при подготовке исходных текстов. В некоторых случаях, например, при генерации, перемещении, удалении исходных текстов в ходе сборки, АК-ВС 3 не может правильно определить состав исходных текстов, это проявляется в ошибках на этапе анализа. В этом случае может потребоваться вручную составить или поправить файлы.

Утилита **akvs-build-monitor** автоматической конфигурации для проектов на C/C++ и Java в среде Linux

Утилита **akvs-build-monitor** перехватывает сборку проектов на Java (если они используют `maven`) и на C/C++. В данной версии **akvs-build-monitor** работоспособна под следующими ОС: Ubuntu 18.04, Ubuntu 20.04, Астра Линукс SE 1.6, 1.7, Debian 10, 11, Cent OS 7, Red OS Murom.

Подготовка к запуску

Для ОС Astra Linux 1.7 нужно подготовить систему, разрешив отладку процессов с помощью `strace`, для этого надо выполнить следующие команды через `sudo`:

```
sudo astra-pttrace-lock disable
```

```
sudo systemctl stop astra-pttrace-lock
```

```
sudo systemctl disable astra-pttrace-lock
```

Далее надо перезагрузить систему и убедиться, что команда

```
cat /proc/sys/kernel/yama/ptrace_scope
```

печатает «0»

Запуск

Утилита находится в директории, где развёрнут дистрибутив АК-ВС для сборочного стенда. Если при установке было выбрано добавлять утилиты в путь, то запуск производится так:

```
akvs-build-monitor [-e] [--strace-log-file имя-файла] -s команда_сборки арг ...
```

Например,

```
akvs-build-monitor -s make
```

Результат работы утилиты – архив **src_config.zip**, а также директория **akvs-build-monitor-out**, в которой сохраняются логи **strace** для процессов, выполнявшихся во время сборки. Ключ **-e** включает печать сообщений **strace** на консоль, а **--strace-log-file** включает вывод сообщений **strace** в заданный файл.

Особенность: очереди сообщений POSIX

В ходе работы утилиты используются очереди сообщений POSIX. Поэтому, если в сборке они тоже используются, то это может повлиять на работу утилиты. Кроме того, если утилита завершилась нештатно (например, была убита), то очередь сообщений может остаться в системе. Чтобы увидеть очереди сообщений в системе, нужно подать команду

```
ipcs -q
```

Оставшуюся очередь нужно удалить, в противном случае следующий запуск может зависнуть. К сожалению, нет способа надёжно

идентифицировать очередь сообщения, относящуюся к монитору. Очередь удаляется командой **ipcrm**.

Известные проблемы: тесты и configure

Тесты и популярный скрипт **./configure**, которые часто входят в сценарии сборки, затрудняют перехват сборки. При возникновении проблем лучше перехватывать сборку без этих этапов.

Ограничения

1. Анализатор АЧКПВ и МКЧА не полностью совместим с gcc < 5.2. При использовании такого компилятора будет выдано предупреждение.
2. Скрипты сборки зачастую генерируют, удаляют, перемещают или переименовывают файлы. В них может применяться **libtool**, в результате чего имя объектного модуля может измениться, производится конкатенация объектных модулей и т.п. Во всех этих случаях монитор не сможет правильно составить файлы конфигурации. Это приведёт к ошибкам при анализе.
3. Анализатор ограничен по масштабу проектов, которые он способен обработать. Например, при анализе ОС Linux не стоит пытаться проанализировать все исходные тексты ОС целиком. Анализируйте отдельные пакеты.
4. Отслеживание линковки происходит только для статической линковки. Линковка разделяемых библиотек не перехватывается и межмодульный анализ через границы разделяемых библиотек не проводится.

Утилита `crrconf_linux.py` автоматической конфигурации для проектов на C/C++ в среде Linux

crrconf_linux.py – это устаревшая версия утилиты автоматической конфигурации. Она может пригодиться на тех ОС, которые не поддерживаются утилитой **akvs-build-monitor**, или в случаях, когда **akvs-build-monitor** по какой-то причине не работает.

В зависимости от выбора, сделанного при установке, утилита **crrconf_linux.py** может быть доступна по пути поиска на сборочном

стенде. Если она не доступна, её можно найти в директории развёртывания программы установки для сборочного стенда.

Скрипт выполняет команду для сборки через утилиту **strace**, которая отслеживает системные вызовы. Лог **strace** записывается в **«/tmp/_cppconfig_log»**, на основе которого скрипт создает архив **«src_config.zip»** в директории запуска.

Примечание. Для работы утилиты требуются установленные программы **strace** 5.7 или выше и **Python** версии 2.7 или 3. Python и **strace** устанавливаются из пакетов во время развёртывания дистрибутива для сборочного стенда.

Алгоритм работы, если проект можно собрать с использованием **strace**, представлен ниже.

Запустите скрипт командой:

cppconfig_linux.py -s команда_сборки

Пример:

cppconfig_linux.py -s make

Результат работы утилиты – архив **src_config.zip** с исходным текстом проекта, конфигурационным файлом и используемыми сторонними заголовочными файлами, который для анализа нужно загрузить на сервер АК-ВС 3.

1. Анализатор АЧКПВ и МКЧА не полностью совместим с **gcc** < 5.2. При использовании такого компилятора будет выдано предупреждение.
2. Скрипты сборки зачастую генерируют, удаляют, перемещают или переименовывают файлы. В них может применяться **libtool**, в результате чего имя объектного модуля может измениться, производится конкатенация объектных модулей и т.п. Во всех этих случаях монитор не сможет правильно составить файлы конфигурации. Это приведёт к ошибкам при анализе.

3. Анализатор ограничен по масштабу проектов, которые он способен обработать. Например, при анализе ОС Linux не стоит пытаться проанализировать все исходные тексты ОС целиком. Анализируйте отдельные пакеты.

4. Отслеживание линковки происходит только для статической линковки. Линковка разделяемых библиотек не перехватывается и межмодульный анализ через границы разделяемых библиотек не проводится.

Утилита автоматической конфигурации для проектов на C/C++ в среде Windows

Для создания архива с конфигурацией проекта на языках C/C++ в среде Windows используется утилита **CppConfMonitor**, которая поставляется в виде архива **monitor.zip**. Архив содержит x64 и x86 версии, использовать x86 на 64-битных системах не рекомендуется.

Для функционирования программа требует права администратора, так как использует системные средства ОС Windows.

Для перехвата процессов и обнаружения прочтённых файлов **CppConfMonitor** использует вспомогательные исполняемые файлы **EtwProcessor** и **ProcessCatcher**. При копировании или перемещении основной утилиты они также должны быть скопированы/перемещены с сохранением относительности путей.

CppConfMonitor может быть запущен отдельно, и тогда необходимо его вручную остановить после завершения сборки, отправив «S» в консоль. Либо можно по аналогии с **cppconf_linux.py** указать команду сборки, тогда мониторинг будет остановлен автоматически.

Например:

CppConfMonitor.exe --directory C:\MyExe\ -e make

Если утилите не указывать папку проекта, то для анализа будут собраны только файлы, для которых были вызваны процессы компиляции. Таким образом, большинство `template`, `inline`-функций и включённых файлов

исходных текстов не будут проанализированы сигнатурным анализатором.

Если указать папку проекта (флаг **-p** или **--project**), то все прочтённые файлы, имеющие расширения C/C++ файлов исходных текстов или заголовочных файлов и находящиеся внутри папки с проектом, будут добавлены для анализа.

Например, в данном случае условные **C:\MyExe\1.h** и **C:\MyExe\2.h** тоже будут проанализированы, хотя компилировался только **1.cpp**:

CppConfMonitor.exe -p C:\MyExe\ --directory C:\MyExe\ -e make

В программе присутствует флаг **--old_monitoring**, который позволяет использовать другой механизм обнаружения процессов компиляции, но пользоваться им рекомендуется только в крайнем случае, так как он имеет небольшую вероятность пропуска или помехи выполнению некоторых из обнаруживаемых процессов. В первом случае будет выведена ошибка в консоли монитора, во втором случае прервётся процедура сборки. Особенно свойственно такое поведение для проектов с большим количеством быстрых процессов компиляции.

Ручная настройка

Для проведения “ручной” конфигурации необходимо понимать структуру конфигурационного файла проекта АК-ВС 3 и знать о некоторых параметрах, передаваемых компиляторам.

Формат конфигурационного файла

Конфигурационный файл - это архив в формате zip, где

- ❖ **parser-cpp** (без расширения) - файл с перечнем команд компиляции;
- ❖ **relations-cpp** (без расширения) - файл с описанием связей между исходными текстами и выполняемыми модулями, нужен только для МКЧА и АЧКПВ;
- ❖ заголовочные файлы.

Файл **parser-cpp** – это XML-файл следующего вида:

Корневой элемент - **config**, дочерние элементы:

- ❖ **settings** - хранит общие настройки парсера в виде линейного списка из узлов **param**;
- ❖ **compilerConfigs** - хранит опции парсера для компиляторов и стандартов языка C++ в виде линейного списка из **macro** и **idir**;
- ❖ **arguments** - хранит аргументы парсера для отдельных файлов в виде линейного списка из **fileArgs**.

Общие настройки парсера

Каждая настройка хранится в теге **param**, значение атрибута **name** задает название параметра, **value** - значение. Пример:

```
<settings>
  <param name= "threadCount" value= "0" />
  <param name= "usePredefines" value= "0" />
</settings>
```

Доступные параметры приведены в таблице:

Название	Описание	Возможные значения	Значение по умолчанию
threadCount	количество потоков	• -1 программа сама вычисляет необходимое количество; • 0 - не использовать потоки; • $n(n>0)$ - количество потоков	-1
errorsLimit	Количество обрабатываемых ошибок, при превышении – весь файл считается ошибочным	• -1 все ошибки; • 0 – не допускать ошибок; • $n>0$ – допускать n ошибок	5
skipBuiltinErrors	Игнорировать ошибки, содержащие текст "use of undeclared identifier '__builtin_'"	• 0 – нет; • 1 – да.	1
useAutoTypeForReturn	использование типа «auto» для временной переменной выражения под return	• 0 - не использовать (вычисляется возвращаемый тип функции); • 1 - использовать	0
usePredefines	Загрузка предустановленных макросов ОС и компилятора	• 0 - не загружать; • 1 - загружать	1
insertProbeInUnknown-Encoding	Вставлять датчики в файлы, кодировку которых не удалось определить	• 0 – не вставлять; • 1 – вставлять	0
insertProbesIfError	Вставлять датчики в файлы, при анализе которых возникли ошибки разбора	• 0 — не вставлять; • 1 — вставлять, игнорируя ошибки заголовочных файлов; • 2 — вставлять.	0

Опции парсера для компиляторов и стандартов языка C++

С помощью опциональных узлов **macro** и **idir** можно задать макросы препроцессора и дополнительные пути заголовочных файлов

❖ **macro**:

- **compiler** — путь до компилятора;
- **std** — используемый стандарт; если не указать, будет использован стандарт по умолчанию;
- **value** — значение, формата **"-DNAME=VALUE"**.

Пример:

```
<macro compiler="/usr/bin/c++" [std="c++11"] value="-DFOO=BAR" />
```

❖ **idir**:

- **compiler** — путь до компилятора;
- **std** — используемый стандарт, если не указать будет использован стандарт по умолчанию;
- **value** — значение, формата **"-I../prj/config/include/path"**.

Пример:

```
<idir compiler="/usr/bin/c++" [std="c++11"] value="-I../prj/config/usr/include" />
```

Аргументы парсера для отдельных файлов

Состоит из линейного списка **fileArgs**, представляет собой список компиляций исходного текста с одним путём.

Через аргументы можно задать путь к заголовочным файлам, директивы препроцессора, язык, компилятор, порядок обработки ошибок и т.п. Список аргументов схож со списком аргументов **clang** и **gcc**.

Аргументы задаются с помощью тегов следующим образом:

❖ **fileArgs**:

- **path** — путь к файлу с исходными текстами.

Тело тега состоит из линейного списка тегов **<args>**.

❖ **args:**

- **compiler** — путь до компилятора;
- **std** — используемый стандарт; если не указать, будет использован стандарт по умолчанию.

Тело элемента состоит из линейного списка тегов **<argument>**.

❖ **argument:**

- **value** — аргумент компилятора

Часто используются следующие:

- **-I** — путь до заголовочного файла;
- **-D** — макрос.

Пример:

```
<fileArgs path="home/user/foo.c">
  <args compiler="/usr/bin/c++">
    <argument value="-I../prj/config/usr/include/a" />
    <argument value="-DFOO=BAR" />
  </args>
  <args compiler="/usr/bin/c++" std="c++11">
    <argument value="-I../prj/config/usr/include/b" />
    <argument value="-DBAR=FOO" />
    <argument value="-ferror-limit=0"/>
  </args>
</fileArgs>
```

Заголовочные файлы могут как принадлежать проекту, так и не принадлежать (например, системные заголовочные файлы, заголовочные файлы от сторонних библиотек).

Если заголовочный файл есть в архиве с исходными текстами и конфигурацией (**src_config.zip**), то достаточно просто указать путь до него — путь задается относительно корня архива **src_config.zip**.

Если заголовочного файла нет в архиве с исходными текстами и конфигурацией (**src_config.zip**), то его надо добавить в **src_config.zip**,

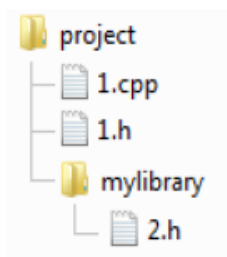
например, в папку **sysinclude**, а так же прописать путь с префиксом «../prj», т.е: «**value="-I../prj/sysinclude/"**».

Заголовочные файлы могут как принадлежать проекту, так и не принадлежать (например, системные заголовочные файлы, заголовочные файлы от сторонних библиотек).

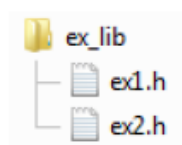
- ❖ Если заголовочный файл есть в архиве с исходными текстами и конфигурацией (**src_config.zip**), то достаточно просто указать путь до него - путь задается относительно корня архива **src_config.zip**.
- ❖ Если заголовочного файла нет в архиве с исходными текстами и конфигурацией (**src_config.zip**), то его надо добавить в **src_config.zip**, например, в папку **sysinclude**, а также прописать путь с префиксом **../prj**, т.е: **value="-I../prj/sysinclude/"**.

Синтетический пример

Есть директория с проектом project:



и директория с заголовочными файлами от сторонней библиотеки:



1.cpp имеет вид:

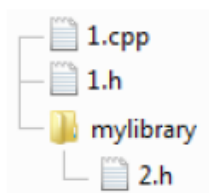
```
1  #include "1.h"
2  #include "2.h"
3  #include "ex1.h"
4  #include "ex2.h"
```

Соберем проект через командную строку:

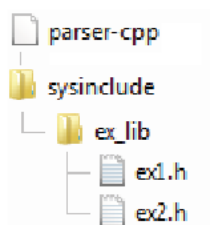
```
g++ -I. -Imylibrary -I/home/akvs/ex_lib/ -DSOMEDEFINE
```

В данном случае мы создаем архив src_config.zip, содержащий два архива, src.zip и prj.zip.

Архив src.zip с содержимым директории project выглядит следующим образом:



Архив prj.zip с содержимым:



Файл с настройкой компилятора будет иметь такой вид:

parser-cpp:

```
<?xml version="1.0"?>
  <config>
    <settings>
    </settings>
    <compilerConfigs>
      <macro compiler="g++" value="-ferror-limit=0" />
    </compilerConfigs>
    <arguments>
      <fileArgs path="1.cpp ">
        <args compiler="g++">
          <argument value="-DSOMEDEFINE" />
          <argument value="-I." />
          <argument value="-Imylibrary" />
          <!-- файлы ex_lib лежат не в src.zip, а в prj.zip, поэтому
добавляем префикс ../prj/ -->
          <argument
            value="I../prj/sysinclude/ex_lib" />
        </args>
      </fileArgs>
    </arguments>
  </config>
```